

Lecture 4

The Structural Design pattern – Bridge pattern

Learning Objectives

By the end of this lecture, students will be able to:

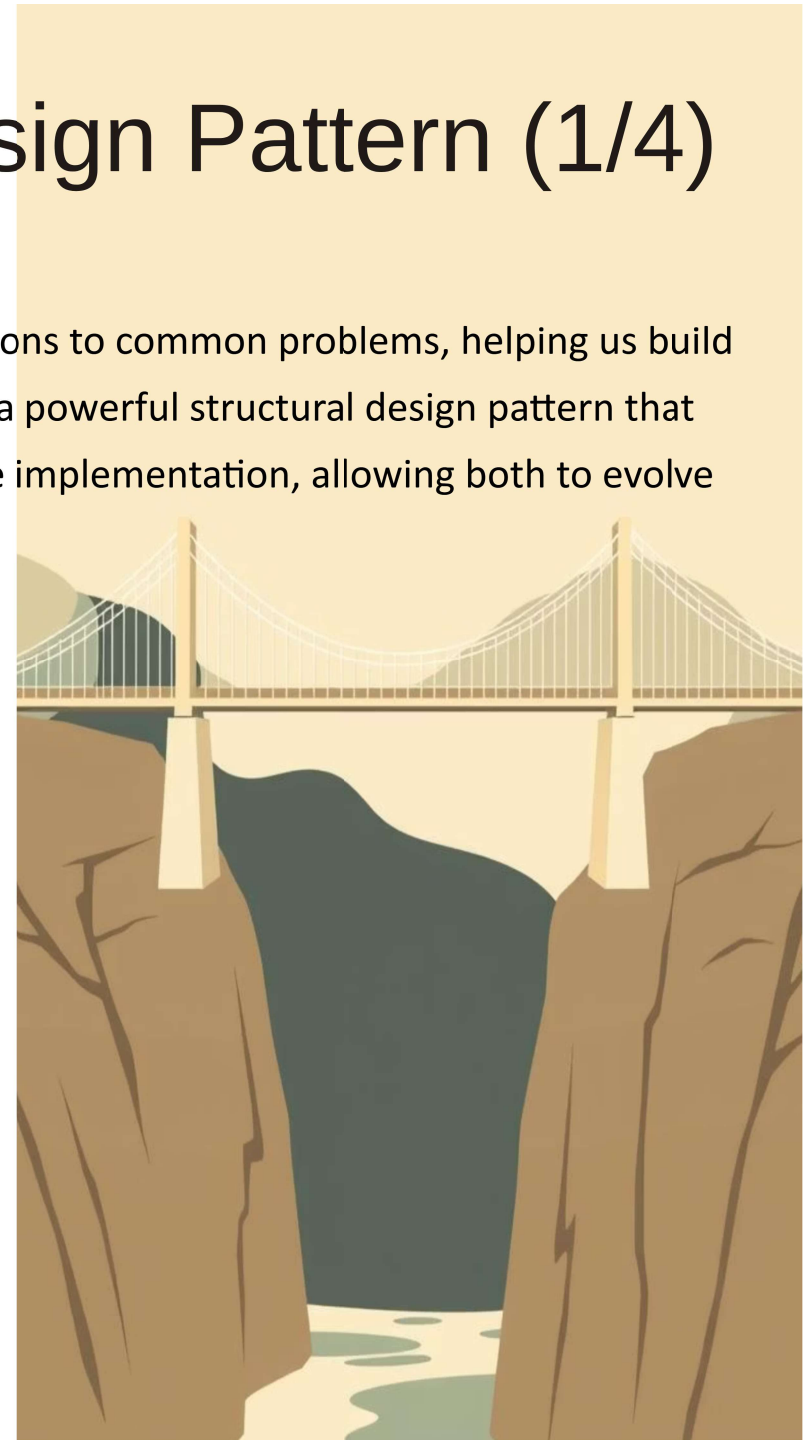
- Understand the intent and structure of the Bridge pattern.
- Know when to use the Bridge pattern.
- Distinguish Bridge from related patterns (e.g. Adapter).
- Write Java code using the Bridge pattern.
- Weigh advantages vs disadvantages.

Introduction to the Bridge Design Pattern (1/4)

In the world of software engineering, design patterns offer proven solutions to common problems, helping us build more flexible, maintainable, and scalable systems. The Bridge pattern is a powerful structural design pattern that enables us to separate the abstract definition of a class from its concrete implementation, allowing both to evolve independently.

Motivation / Problem Statement:

- When you have multiple dimensions of variation.
- Example: Shapes (circle, square) and Colors (red, blue).
- Without Bridge, you may need class combinations: RedCircle, BlueSquare, etc. (combinatorial explosion).
- Want to be able to vary shapes independently of colors.



Introduction to the Bridge Design Pattern (2/4)

The Bridge design pattern is a structural pattern that aims to decouple an abstraction from its implementation, allowing the two to vary independently. This means that changes to the abstraction do not affect the implementation, and vice versa. It's a powerful mechanism for building flexible and extensible software systems.

At its core, the Bridge pattern addresses the challenge of managing class hierarchies that grow exponentially when dealing with multiple dimensions of variation. Instead of using inheritance to combine these dimensions, which leads to a rigid and complex class structure, the Bridge pattern uses composition to link the abstraction and its implementation.

This pattern is particularly useful when you need to choose between different implementations at runtime, or when you want to extend both the abstraction and the implementation without creating a tangled web of subclasses. It promotes loose coupling, making your code easier to understand, test, and maintain.

Introduction to the Bridge Design Pattern (3/4)

When to Use Bridge Pattern

- When both abstractions and implementations are expected to vary.
- When you want to avoid a permanent binding between abstraction and implementation (i.e. avoid combinatorial subclass explosion).
- When you want runtime flexibility: choosing implementations at runtime.
- When you want to separate interface from platform-specific or implementation-specific code (e.g., GUI frameworks, database drivers, rendering engines).

When Not to Use / Drawbacks

- If there is unlikely to be many variations, overhead may not justify complexity.
- Additional layers cause indirection, making code harder to follow.
- Performance overhead (perhaps small) due to extra abstraction.
- Over-engineering if requirements are static.

Introduction to the Bridge Design Pattern (4/4)

Advantages

1. Decoupling Abstraction & Implementation — they can evolve independently.
2. Fewer subclasses / reduced combinatorial explosion.
3. Runtime flexibility: change implementation at runtime.
4. Improved maintainability & extensibility.
5. Better for parallel development: teams can work on both sides separately.

Disadvantages

1. Increased complexity, more classes/interfaces.
2. Indirection makes understanding flow harder.
3. Possible performance/memory overhead.
4. Might be overkill when there are only few variations or static use.

Best Practices & Design Tips

- Identify early whether abstraction and implementation will vary independently.
- Favor interfaces or abstract classes for both sides.
- Use composition, not inheritance, to connect them.
- Keep abstraction side minimal; push implementation details to implementor side.
- Avoid exposing implementor to clients.
- Ensure changes to implementation do not require changes to abstraction (backwards compatibility).

Common Pitfalls / Anti-Patterns

- Overusing bridge when only one implementation exists → unnecessary complexity.
- Letting abstraction leak implementor details to client.
- Tight coupling still creeping in (refined abstraction referring to concrete implementor).
- Ignoring performance impacts in sensitive contexts.

Core Components of the Bridge Pattern

The Bridge design pattern is composed of two main parts: the Abstraction and the Implementation. This separation is key to its flexibility.



Abstraction

This is the high-level control layer. It defines the client-facing interface and contains a reference to an object of the **Implementer** interface. It delegates the actual work to the linked implementation object.



Implementation

This layer defines the interface for the concrete implementation classes. It does not need to correspond directly to the Abstraction's interface and can be very different. The Abstraction uses this interface to perform its operations.

This design mechanism encapsulates an implementation class inside of an interface class. The client code interacts only with the Abstraction, without needing to know the specifics of the underlying Implementation. This allows both parts to be developed and modified independently.

Elements of the Bridge Design Pattern

To fully grasp the Bridge pattern, it's essential to understand its four key elements and how they interact.



Abstraction

The core of the Bridge design pattern. It defines the high-level interface for the client and maintains a reference to an **Implementer** object.



Implementer

Defines the interface for the implementation classes. This interface doesn't have to mirror the Abstraction's interface; it can be entirely different, focusing on primitive operations.



Refined Abstraction

Extends the **Abstraction** and provides more specific or detailed functionality. It hides finer elements from the implementers, focusing on specific variations of the abstraction.

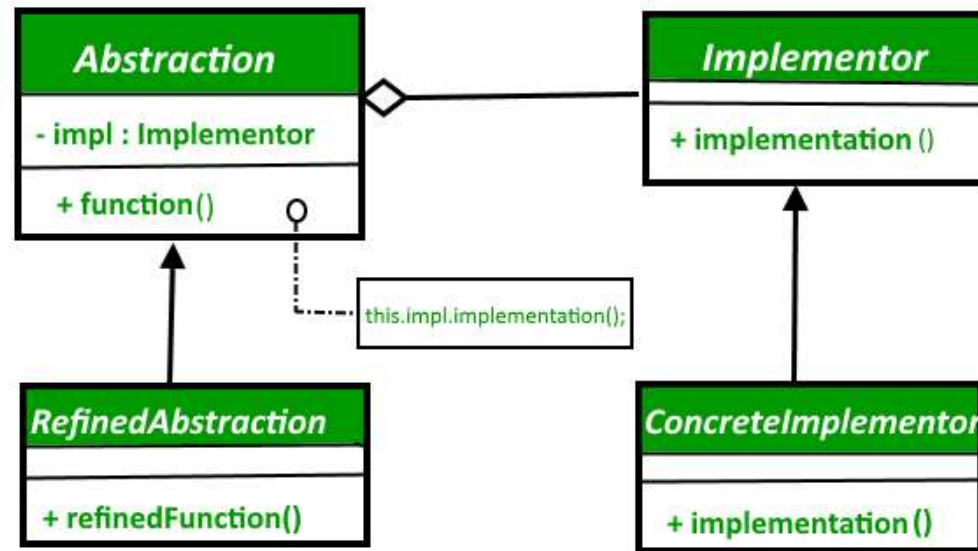


Concrete Implementation

Implements the **Implementer** interface by providing the actual, concrete logic for the operations defined by the Implementer.

This structure ensures that the Abstraction can change its Implementer at runtime, and changes to the Implementer do not affect client code, thereby increasing loose coupling.

UML Diagram: Visualizing the Bridge pattern (1/3)



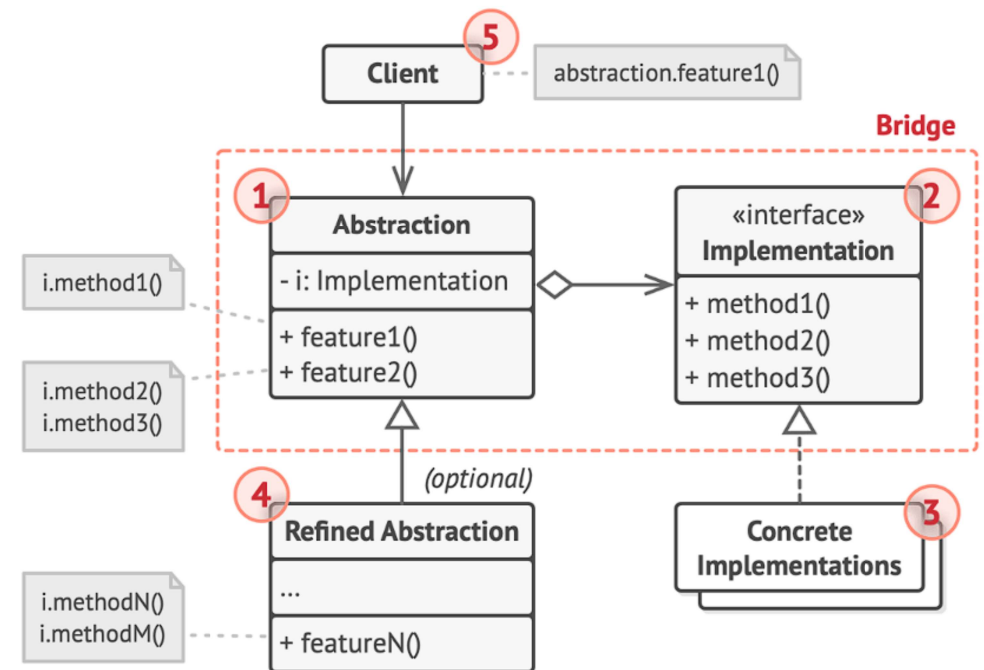
The UML diagram provides a clear visual representation of how the components of the Bridge design pattern interact.

As you can see, the **Abstraction** class holds a reference to the **Implementor** interface. This composition is the "bridge" that connects the two hierarchies. **Refined Abstractions** extend the **Abstraction**, while **Concrete Implementations** implement the **Implementor**. This setup allows for independent development and modification of both sides of the bridge.

UML Diagram: Visualizing the Bridge pattern (2/3)

1. The **Abstraction** provides high-level control logic. It relies on the implementation object to do the actual low-level work.
2. The **Implementation** declares the interface that's common for all concrete implementations. An abstraction can only communicate with an implementation object via methods that are declared here. The **abstraction** may list the same methods as the implementation, but usually the abstraction declares some complex behaviors that rely on a wide variety of primitive operations declared by the implementation.
3. Concrete **Implementations** contain platform-specific code.
4. **Refined Abstractions** provide variants of control logic. Like their parent, they work with different implementations via the general implementation interface.
5. The **Client** is only interested in working with the **abstraction**. However, it's the client's job to link the abstraction object with one of the implementation objects.

Structure

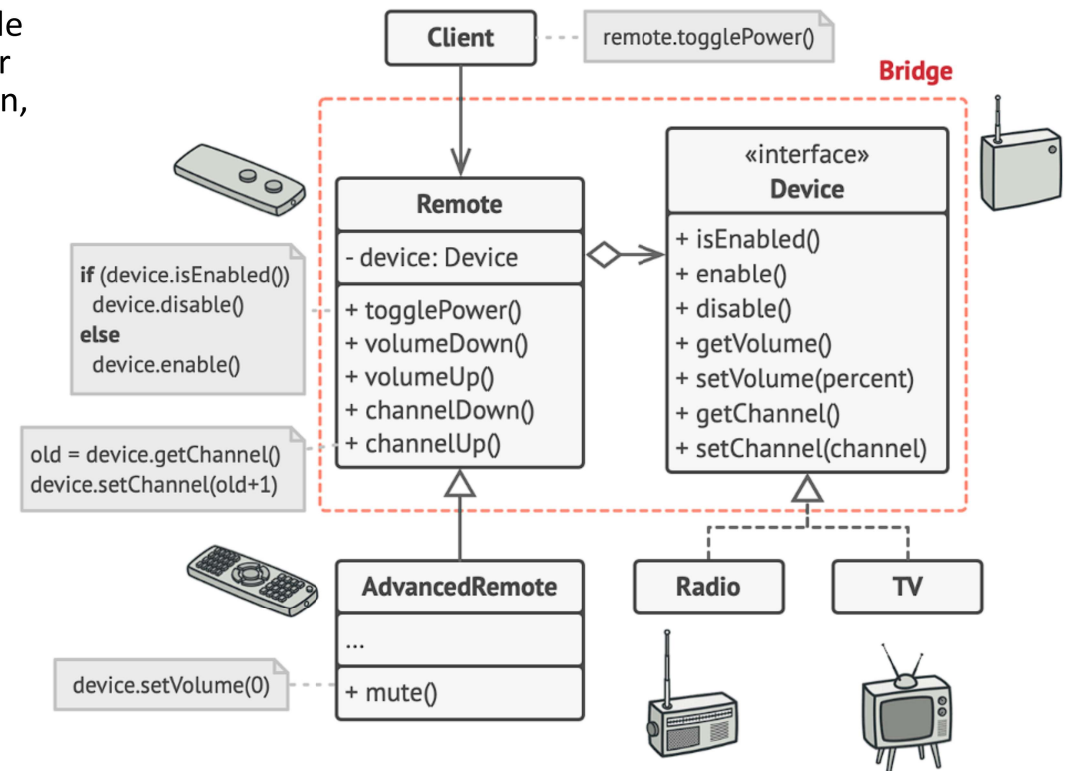


UML Diagram: Visualizing the Bridge pattern (3/3)

Pseudocode

This example illustrates how the Bridge pattern can help divide the monolithic code of an app that manages devices and their remote controls. The Device classes act as the implementation, whereas the Remotes act as the abstraction.

1. The base remote control class declares a reference field that links it with a device object. All remotes work with the devices via the general device interface, which lets the same remote support multiple device types.
2. You can develop the remote control classes independently from the device classes. All that's needed is to create a new remote subclass. For example, a basic remote control might only have two buttons, but you could extend it with additional features, such as an extra battery or a touchscreen.
3. The client code links the desired type of remote control with a specific device object via the remote's constructor.



The original class hierarchy is divided into two parts: devices and remote controls.

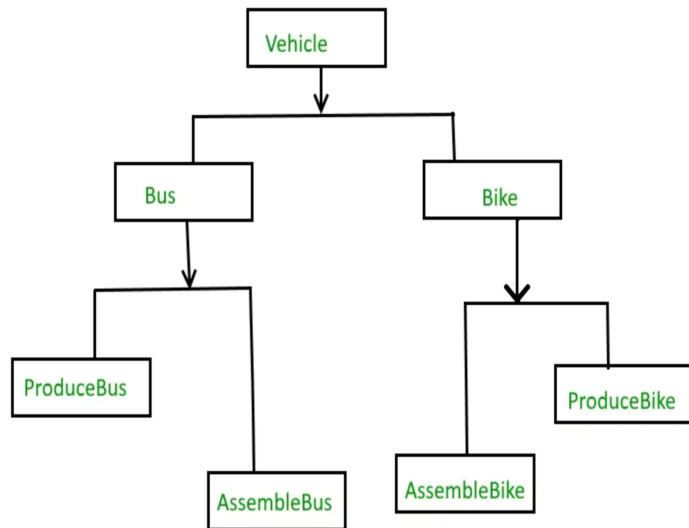
Why Use the Bridge Pattern?

Answer: Prefer Composition Over Inheritance

The Bridge pattern is a practical application of the principle "prefer composition over inheritance." It becomes invaluable when you face scenarios where multiple, orthogonal dimensions of variation exist within your system.

The Problem Without Bridge

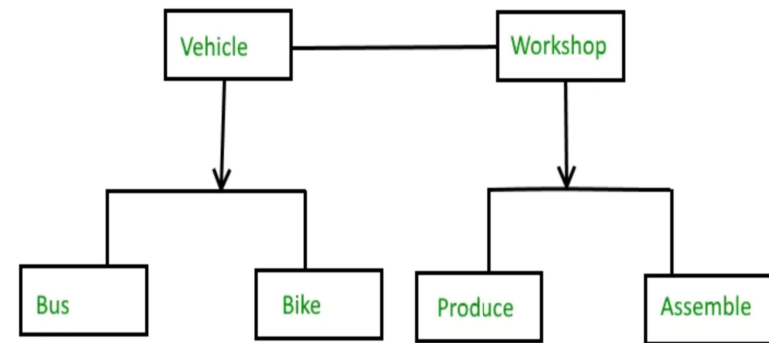
Consider a scenario where you have different types of vehicles (Car, Bike) and different manufacturing workshops (Produce, Assemble). Without the Bridge pattern, you might end up with a class hierarchy like this:



This approach leads to a combinatorial explosion of classes (e.g., ProduceCar, AssembleCar, ProduceBike, AssembleBike). Any change to a vehicle type or a workshop process would require modifications across many classes, making the system rigid and hard to maintain.

The Solution With Bridge

The Bridge pattern decouples these concerns. Vehicle types become abstractions, and workshop processes become implementations.



Here, **Vehicle** (Abstraction) holds a reference to **Workshop** (Implementer). New vehicle types or workshop processes can be added independently without affecting existing code. This decoupling makes the system much more flexible and scalable.

Bridge pattern's interactions with others (1/2)

Comparison: Bridge vs Adapter vs Strategy

<u>Pattern</u>	<u>Intent</u>	<u>When used</u>	<u>Key Difference with Bridge</u>
Bridge	Decouple abstraction and implementation so they vary independently	Multiple orthogonal dimensions of variation; want flexibility and separate hierarchies	Designed from the start for this decoupling; uses composition across hierarchies.
Adapter	Make an existing interface compatible with another expected by clients	Integrating legacy code; mismatched interfaces	Usually one-off, client adapts to an existing interface; not about varying implementations.
Strategy	Define a family of algorithms, encapsulate each one, make them interchangeable	When behavior varies but the interface remains constant	Strategy is about behavior, Bridge is about abstraction-implementation separation.

Bridge pattern's interactions with others (2/2)

- **Bridge pattern** is designed first by letting to develop parts of an application independently of each other whereas **Adapter pattern** is used with an existing app to make some otherwise-incompatible classes work together smoothly.
- **Bridge, State, Strategy and Adapter patterns** have very similar structures. In fact, all of these patterns are based on composition, which is delegating work to other objects. But, all of them solve different problems.
- **Abstract Factory pattern** can be used with **Bridge pattern** together. **Bridge pattern** can work only with specific implementations where Abstract Factory can encapsulate these relations and hide the complexity from the client code.
- It's possible to combine **Builder pattern** with **Bridge pattern**: the director class takes the role of the abstraction, while different builders act as implementations.

Code examples of the Bridge pattern components

// EXAMPLE # 1

// Implementor

```
public interface DrawAPI { void drawCircle(int radius, int x, int y); }
```

```
public class RedCircle implements DrawAPI { public void  
drawCircle(int r, int x, int y) { System.out.println("Red circle"); } }
```

// Abstraction

```
public abstract class Shape { protected DrawAPI drawAPI; protected  
Shape(DrawAPI api) { this.drawAPI = api; } public abstract void  
draw(); }
```

```
public class Circle extends Shape {  
    public Circle(int r, int x, int y, DrawAPI api) { super(api); /* store  
r,x,y */ }  
    public void draw() { drawAPI.drawCircle(/* params */); }  
}
```

// Usage

```
Shape redCircle = new Circle(100, 100, 10, new RedCircle());  
redCircle.draw();
```

// EXAMPLE # 2

// Implementor interface

```
interface Renderer { void renderCircle(double x, double y, double  
radius); }
```

```
class VectorRenderer implements Renderer {  
    public void renderCircle(double x, double y, double radius) {  
        System.out.println("Drawing circle as vector at " + x + "," + y);  
    }  
}
```

```
class RasterRenderer implements Renderer {  
    public void renderCircle(double x, double y, double radius) {  
        System.out.println("Drawing circle as pixels at " + x + "," + y);  
    }  
}
```

// Abstraction

```
abstract class Shape {  
    protected Renderer renderer;  
    protected Shape(Renderer renderer) { this.renderer = renderer; }  
    public abstract void draw();  
}
```

```
class Circle extends Shape {  
    private double x, y, radius;  
    public Circle(double x, double y, double radius, Renderer  
renderer) {  
        super(renderer); this.x = x; this.y = y; this.radius = radius;  
    }  
    public void draw() { renderer.renderCircle(x, y, radius); }  
}
```

// Client usage

```
Renderer vector = new VectorRenderer();  
Shape circle = new Circle(5, 5, 10, vector);  
circle.draw(); // draws as vector
```

Practical Example: Vehicle Manufacturing

```
// Abstraction: Defines the vehicle and holds a reference to workshops
```

```
abstract class Vehicle {  
    protected Workshop workShop1;  
    protected Workshop workShop2;  
    protected Vehicle(Workshop workShop1, Workshop workShop2) {  
        this.workShop1 = workShop1;  
        this.workShop2 = workShop2;  
    }  
    abstract public void manufacture();  
}
```

```
// Refined Abstraction 1: Car
```

```
class Car extends Vehicle {  
    public Car(Workshop workShop1, Workshop workShop2) {  
        super(workShop1, workShop2);  
    }  
}
```

```
@Override
```

```
public void manufacture() {  
    System.out.print("Car ");  
    workShop1.work(); // Delegate to workshop 1  
    workShop2.work(); // Delegate to workshop 2  
}}
```

```
// Refined Abstraction 2: Bike
```

```
class Bike extends Vehicle {  
    public Bike(Workshop workShop1, Workshop workShop2) {  
        super(workShop1, workShop2);  
    }  
}
```

```
@Override
```

```
public void manufacture() {  
    System.out.print("Bike ");  
    workShop1.work(); // Delegate to workshop 1  
    workShop2.work(); // Delegate to workshop 2  
}}
```

Let's illustrate the Bridge pattern with a concrete Java example of vehicle manufacturing, where we separate vehicle types from their production processes.

Practical Example: Workshop Implementations

```
// Implementer: Defines the interface for workshop processes

interface Workshop {
    abstract public void work();
}

// Concrete Implementation 1: Produce
class Produce implements Workshop {
    @Override
    public void work() {
        System.out.print("Produced"); }}

// Concrete Implementation 2: Assemble
class Assemble implements Workshop {
    @Override
    public void work() {
        System.out.print(" And");
        System.out.println(" Assembled.");}

// Demonstration of Bridge Design Pattern
class BridgePattern {
    public static void main(String[] args) {

// Create a Car with Produce and Assemble workshops
        Vehicle vehicle1 = new Car(new Produce(), new
Assemble());
        vehicle1.manufacture(); // Output: Car Produced And
Assembled.

// Create a Bike with Produce and Assemble workshops
        Vehicle vehicle2 = new Bike(new Produce(), new
Assemble());
        vehicle2.manufacture(); // Output: Bike Produced And
Assembled. }}
```

Continuing our vehicle manufacturing example, here are the Implementer interface and its concrete implementations.

In this example, we can easily add new vehicle types (e.g., Truck) or new workshop processes (e.g., Paint, Test) without modifying the existing Vehicle or Workshop classes. This demonstrates the power of decoupling through the Bridge pattern.

Key Advantages of the Bridge Design Pattern

The Bridge pattern offers several significant benefits that contribute to more robust and maintainable software architectures.

Decoupling Abstraction and Implementation

This is the primary advantage. It allows both the abstract definition and its concrete implementation to vary independently, preventing a tight coupling that can lead to rigid systems.

Platform Independence

It is widely used for implementing features that need to be independent of the underlying platform or specific technologies. For example, drawing shapes on different graphics APIs.

Increased Flexibility and Extensibility

You can extend both the abstraction and the implementation hierarchies separately. Adding new functionalities to either side doesn't require changes to the other, promoting open/closed principle.

Run-time Binding of Implementation

The Bridge pattern allows you to change the implementation used by an abstraction at runtime, providing dynamic behavior and adaptability.

Reduced Code Duplication

By separating concerns, it helps avoid the creation of many similar classes that would arise from combining multiple dimensions of variation through inheritance.

Summary and Key Takeaways

The Bridge Design Pattern is a powerful tool for building flexible and maintainable software. It's all about separating "what something does" from "how it does it."



Structural Pattern

Focuses on composing classes and objects to form larger structures, making them more flexible and efficient.



Decoupling

Its core purpose is to decouple an abstraction from its implementation, allowing them to vary independently.



Composition Over Inheritance

A prime example of preferring composition to manage orthogonal class hierarchies, avoiding class explosion.



Flexibility & Extensibility

Enables adding new abstractions or implementations without modifying existing code, adhering to the Open/Closed Principle.



Runtime Adaptability

Allows changing the implementation at runtime, providing dynamic behavior.

By mastering the Bridge pattern, you gain a valuable technique for designing software that is adaptable, scalable, and easier to evolve over time. Remember to consider it when you encounter situations with multiple, independent dimensions of variation.